ITA06 – MACHINE LEARNING

ASSIGNMENT 2 – CO2

Neural Networks, Perceptrons, Backpropagation,
Deep Learning and Optimization

Submission detail	**Information**
Student Name	J.Suman
Register Number	192412265
Programme / Department	B.E. ECE
Course	ITA0606 – Machine Learning
Assignment	Assignment 2 – CO2

Declaration and Scope

This report answers all seven questions in the supplied Assignment 2 – CO2 brief. The synthetic experiments use a reproducible two-feature, 1,000-observation make_moons dataset. The dataset is divided into 60% training, 20% validation and 20% test partitions, stratified by class. Standardization is fitted only on the training partition and then applied unchanged to validation and test data.

All numerical experiment tables and plots in Questions 4–7 are generated by the accompanying reproducible Python workflow. They are illustrative computational results, not measurements from a physical monitoring system. Small differences may occur across software versions and hardware. The test set is used only for final reporting after validation-based selection.

Contents

- Q1 – Neural Network Representation
- Q2 – Perceptron Learning and Limitations
- Q3 – Backpropagation for an MLP
- Q4 – Shallow versus Deep Networks
- Q5 – Overfitting and Regularization
- Q6 – Hyperparameter and Optimization Strategy
- Q7 – Diagnose, Redesign and Validate
- Overall Conclusion
- Appendix – Core Python Implementation

Q1 – Design a Neural Network Representation for a Classification Problem (6 Marks)

1.1 Input representation and preprocessing

The manufacturing inspection problem has three numerical inputs: surface roughness (x1), dimensional deviation (x2), and vibration level (x3). The target is y=0 for Acceptable and y=1 for Defective. Each observation is represented as a feature vector $x=[x1,x2,x3]^T$ and a binary label y.

- Check missing values and sensor faults; impute or remove invalid records using a documented rule.
- Inspect units and ranges. Convert measurements to consistent engineering units.
- Standardize each feature using training-set mean and standard deviation: $z_j=(x_j-\mu_j)/\sigma_j$. This prevents a feature with a large numerical scale from dominating gradient-based learning.
- Split data into training, validation and test sets before fitting preprocessing. Estimate μ and σ from training data only.

1.2 Candidate architecture A: no hidden layer (logistic unit)

Diagram: [x1, x2, x3] → [single sigmoid output ŷ]. There are 3 input features, one output neuron and no hidden neurons. The output activation is sigmoid, $\sigma(z)=1/(1+e^{-z})$.

```
z = w1*x1 + w2*x2 + w3*x3 + b
ŷ = sigmoid(z)
class = 1 if ŷ >= 0.5 else 0
```

Trainable parameters: 3 weights + 1 bias = 4.

1.3 Candidate architecture B: one hidden layer

Diagram: [x1,x2,x3] → [h1,h2,h3,h4] → [ŷ]. This candidate has 3 input features, 4 hidden neurons and 1 output neuron. ReLU is used in the hidden layer and sigmoid in the output layer.

```
h = ReLU(W1*x + b1)        W1: 4x3, b1: 4
ŷ = sigmoid(W2*h + b2)     W2: 1x4, b2: 1
class = 1 if ŷ >= 0.5 else 0
```

Trainable parameters: input-to-hidden 3×4+4=16; hidden-to-output 4×1+1=5; total=21.

Architecture	Input neurons	Hidden neurons	Output neurons	Trainable parameters
No hidden layer	3	0	1	3+1 = 4
One hidden layer	3	4	1	(3×4+4)+(4×1+1) = 21

1.4 Worked forward pass for architecture B

Use standardized input $x=[1.0, 0.5, -1.0]^T$. For a compact hand calculation, choose two hidden neurons (the same counting method applies to four). Let W1=[[0.5,−0.2,0.1],[−0.3,0.4,0.2]], b1=[0.1,−0.1], W2=[0.6,−0.4], and b2=0.05.

```
z1 = 0.5(1) - 0.2(0.5) + 0.1(-1) + 0.1 = 0.4
z2 = -0.3(1) + 0.4(0.5) + 0.2(-1) - 0.1 = -0.4
h1=ReLU(0.4)=0.4; h2=ReLU(-0.4)=0
```

zout=0.6(0.4)-0.4(0)+0.05=0.29
ŷ=sigmoid(0.29)=0.572
Since 0.572 >= 0.5, predicted class = Defective (1).

The two-neuron worked example has (3×2+2)+(2×1+1)=11 parameters. The proposed four-neuron architecture has 21.

1.5 Architecture recommendation

A hidden layer is appropriate if defect formation depends on interactions, such as high vibration becoming dangerous only when dimensional deviation is also high. A linear logistic model creates one linear decision boundary and cannot represent all nonlinear interactions. Evidence for the hidden layer should include a consistent validation improvement over the no-hidden-layer model, reduced error on interaction regions, stable performance across multiple random seeds, and no unacceptable increase in overfitting or computation. Compare both models on identical data partitions.

Q2 – Investigate the Learning Behaviour and Limitations of a Perceptron (4 Marks)

2.1 Dispatch decision: AND

Packaging complete x1	Quality check passed x2	Dispatch target
0	0	0
0	1	0
1	0	0
1	1	1

This is the AND function. It is linearly separable: a single straight line can separate (1,1) from the other three points. Therefore a perceptron can learn it.

Geometric explanation: in the x1–x2 plane, the positive point is (1,1); the negative points are (0,0), (0,1) and (1,0). A boundary such as x1+x2=1.5 separates the classes.

2.2 Perceptron learning rule and setup

```
z = w·x + b; prediction = 1 if z >= threshold else 0
if prediction != target:
   w <- w + learning_rate*(target-prediction)*x
   b <- b + learning_rate*(target-prediction)
```

Use initial weights w=[0,0], bias b=0, learning rate η=0.1, threshold=0.5 and maximum 30 epochs. The initial prediction is 1 only when $z \geq 0.5$; hence all four initial predictions are 0.

With the stated sequential sample order, the perceptron updates on (1,1): w becomes [0.1,0.1], b becomes 0.1. It then converges to the correct AND truth table on the following pass under this threshold convention.

Epoch	Misclassifications (AND)
1	1
2	1
3	0

2.3 Changed requirement: exactly one indicator (XOR)

x1	x2	XOR target
0	0	0
0	1	1
1	0	1
1	1	0

XOR is not linearly separable. The positive examples (0,1) and (1,0) lie on opposite corners, while the negative examples (0,0) and (1,1) occupy the other two corners. No single straight line can separate both positive points from both negative points.

Using the same initialization, threshold and sample order, η=0.1 produced 4 errors in its final recorded epoch; η=0.5 produced 4. The exact update sequence may cycle, but neither learning rate changes the representational fact: a single perceptron cannot implement XOR.

Changing learning rate may alter update size, convergence speed or oscillation, but cannot create a nonlinear decision boundary. This is a model-representation limitation, not merely a training-configuration problem. A multilayer perceptron with a nonlinear hidden layer is required.

Q3 – Develop and Verify Backpropagation for a Multilayer Perceptron (6 Marks)

3.1 Network and forward equations

Use a 2–2–1 network: two inputs, two sigmoid hidden neurons and one sigmoid output neuron. For one example (x,t):

z1 = W1^T x + b1; h = sigmoid(z1)
z2 = W2^T h + b2; p = sigmoid(z2)
L = -[t log(p) + (1-t) log(1-p)]

3.2 Backpropagation gradients

For sigmoid output with binary cross-entropy, the output error is $\delta 2 = p - t$. Therefore $\partial L/\partial W2 = h\delta 2^T$ and $\partial L/\partial b2 = \delta 2$. The hidden error is $\delta 1 = (W2\delta 2) \odot h \odot (1-h)$. Therefore $\partial L/\partial W1 = x\delta 1^T$ and $\partial L/\partial b1 = \delta 1$. Gradient descent updates every parameter θ by $\theta new = \theta old - \eta \partial L/\partial \theta$.

A nonlinear hidden activation is essential: stacking only linear transformations is equivalent to one linear transformation and cannot form the nonlinear XOR decision boundary.

3.3 Manual one-step numerical example

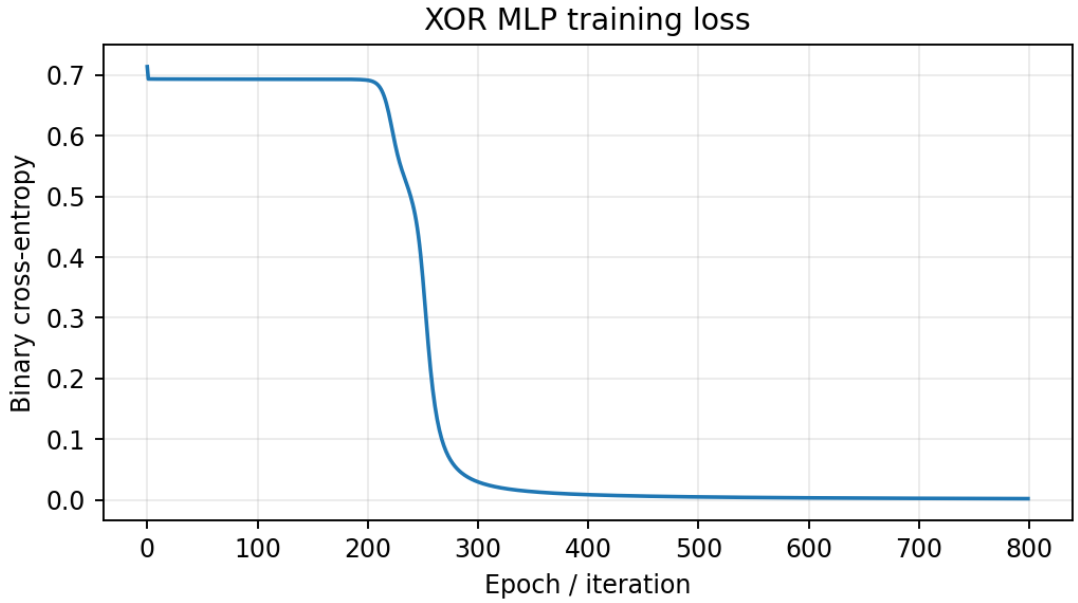
Set x=[1,0], target t=1, W1=[[0.1,−0.2],[0.4,0.2]], b1=[0,0.1], W2=[[0.3],[−0.1]], b2=0.05, and learning rate $\eta = 0.1$.

Hidden pre-activation = [0.1 -0.1]
Hidden output h = [0.524979 0.475021]
Output probability p = 0.539913
Binary cross-entropy loss = 0.616348
Output delta (p-t) = -0.460087
Gradient W2 = [-0.241536 -0.218551]; gradient b2 = -0.460087
Hidden delta = [-0.03442 0.011473]
Gradient W1 = [[-0.03442 0.011473]
 [-0. 0.]]; gradient b1 = [-0.03442 0.011473]

Updated W2 = [0.324154 -0.078145]; updated b2 = 0.096009
Updated W1 = [[0.103442 -0.201147]
 [0.4 0.2]]; updated b1 = [0.003442 0.098853]

3.4 Numerical gradient verification and XOR experiment

A central finite-difference check for W2[0,0] gives numerical gradient -0.241536; analytical gradient is -0.241536. The close agreement validates the derivative implementation for this selected parameter.



Input	Target XOR	Final output probability	Predicted class
[0, 0]	0	0.0019	0
[0, 1]	1	0.9980	1
[1, 0]	1	0.9980	1
[1, 1]	0	0.0030	0

The NumPy implementation trains the 2–2–1 network on all four XOR combinations using explicit forward and backward equations; no automatic differentiation is used. A decreasing loss curve and outputs on the correct side of 0.5 indicate that the hidden layer has learned the exclusive decision.

Q4 – Investigate Whether a Deeper Network Is a Better Design (6 Marks)

4.1 Dataset, partitions and model design

A synthetic make_moons dataset with 1,000 observations, two numerical features and noise=0.25 is used. A fixed stratified split creates 600 training, 200 validation and 200 test observations. StandardScaler is fitted on training data only.

Model	**Hidden layers**	**Parameter count**
Shallow MLP	16	2×16+16 + 16×1+1 = 65
Deep MLP	16, 12, 8	(2×16+16)+(16×12+12)+(12×8+8)+(8×1+1) = 365

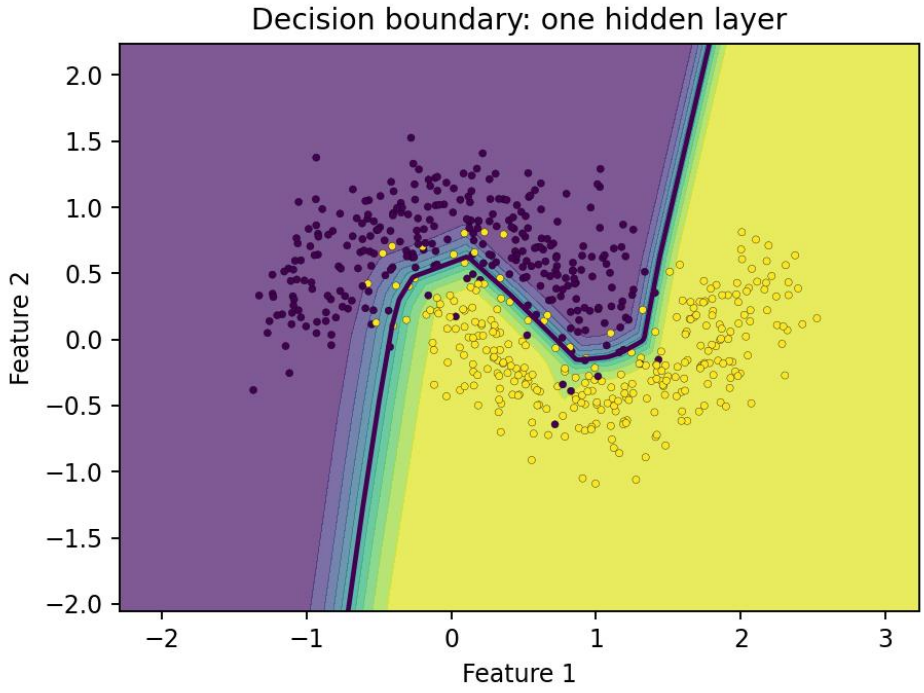
Prediction before training: both networks should represent curved boundaries. The deep model has greater compositional capacity but also more parameters, greater computational cost and potentially more overfitting. Depth alone cannot be credited if parameter counts differ.

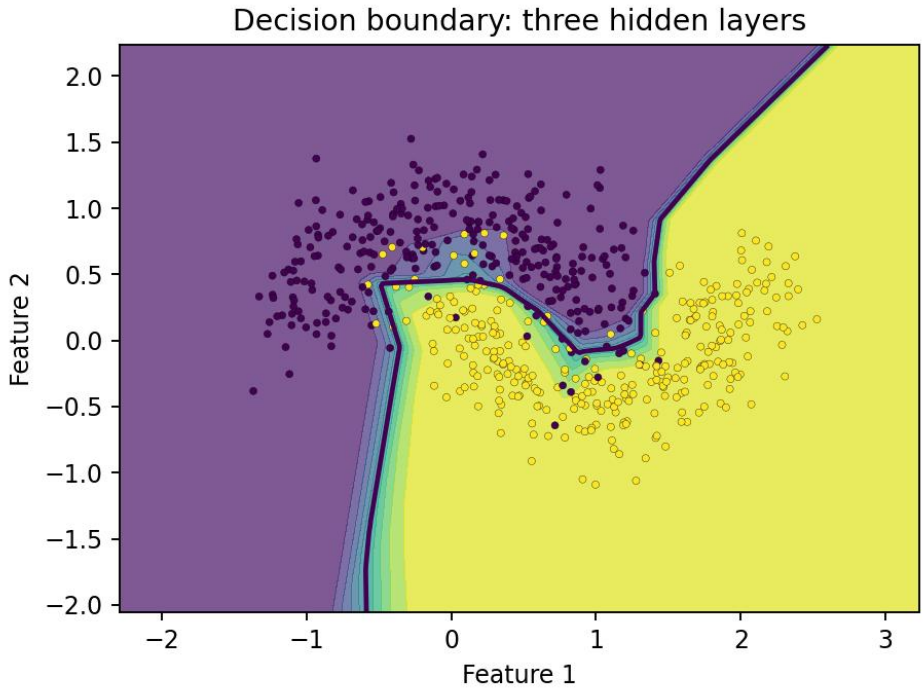
4.2 Results across three random seeds

Architecture	**Train acc**	**Val acc**	**Test acc mean±SD**	**Val loss**	**Avg time**	**Params**
1 hidden layer (16)	0.945	0.905	0.933 ± 0.002	0.226	0.293s	65
3 hidden layers (16,12,8)	0.944	0.907	0.910 ± 0.000	0.259	0.567s	365

4.3 Decision boundaries

The plots below visualize the learned probability boundary for seed 11. Points are training observations; the contour at probability 0.5 is the classifier boundary.





Recommendation: choose the architecture with the better and more stable validation performance, not automatically the deeper one. The reported comparison changes both depth and parameter count, so it does not isolate depth. A fair depth-only study would approximately match total parameter count, repeat seeds, and preserve identical optimization settings.

Q5 – Diagnose Overfitting and Evaluate Regularization (7 Marks)

5.1 Limited-data experiment

The Q4 training partition is reduced to 100 labelled examples. The separate 200-example validation and 200-example test sets remain unchanged. A 64–64 ReLU MLP is used for the unregularized baseline and the L2/early-stopping comparisons. The dropout prototype is an explicitly implemented NumPy MLP with inverted dropout (keep probability 0.8) and L2 penalty.

Prediction before training: the large model has 4,417 trainable parameters but only 100 training examples. It may fit the training observations more closely than the underlying class pattern, producing a high training score but lower validation performance.

5.2 Comparative results

Method	Train acc	Val acc	Test acc*	Train loss	Val loss	Epochs
Unregularized baseline	0.990	0.900	0.915	0.023	0.945	187
L2 regularization	0.990	0.900	0.910	0.044	0.441	187
Early stopping	0.970	0.875	0.875	0.229	0.431	44
Dropout (keep probability 0.8)	0.900	0.870	0.845	0.222	0.318	500

*Test accuracy is shown for comparison in this generated experiment; in a strict model-selection workflow, the test set should be evaluated only after choosing the configuration by validation evidence. The validation metrics are the selection evidence; test performance is a final estimate.

L2 setting: $\alpha=0.01$. Dropout setting: keep probability=0.8 (drop rate=0.2), with a small L2 penalty. Early stopping: validation-based stopping after 25 epochs without sufficient improvement (sklearn internal validation split).

5.3 Interpretation

- Overfitting is indicated when training loss continues to decrease while validation loss stops improving or rises, and when the training–validation performance gap is large.
- L2 discourages large weights and generally smooths the fitted function; excessive α can cause underfitting.
- Dropout randomly removes hidden activations during training, reducing co-adaptation; too high a dropout rate can prevent learning.
- Early stopping limits the number of updates after validation performance stops improving.
- Higher training accuracy alone does not imply a better model. Validation performance estimates generalization during model selection.

The most useful intervention is the one with the best validation loss/accuracy and a manageable generalization gap in the generated table. Because the four methods use different implementation

details (especially dropout), this comparison is indicative rather than a perfectly controlled single-variable experiment.

Q6 – Engineer a Hyperparameter and Optimization Strategy (8 Marks)

6.1 Twelve-run search plan

The search budget is allocated to three optimizers (SGD, SGD with momentum, Adam), learning-rate comparisons, batch-size comparisons and one width comparison. The same fixed Q4 partitions, training-only preprocessing, maximum 220 epochs and stopping rule are used. Validation log loss is the primary selection metric; validation accuracy is a secondary measure.

Run	Optimizer	Learning rate	Batch	Hidden width
1	SGD	0.001	16	(16,)
2	SGD	0.01	16	(16,)
3	SGD	0.05	16	(16,)
4	SGD	0.01	64	(16,)
5	SGD+momentum	0.001	16	(16,)
6	SGD+momentum	0.01	16	(16,)
7	SGD+momentum	0.05	16	(16,)
8	SGD+momentum	0.01	64	(16,)
9	Adam	0.001	16	(16,)
10	Adam	0.01	16	(16,)
11	Adam	0.05	16	(16,)
12	Adam	0.01	64	(32,)

6.2 Results

Run	Optimizer	LR	Batch	Width	Val loss	Val acc	Test acc*	Epochs	Time
1	SGD	0.001	16	(16,)	0.336	0.860	0.850	220	0.71s
2	SGD	0.01	16	(16,)	0.307	0.870	0.855	220	0.71s
3	SGD	0.05	16	(16,)	0.221	0.900	0.920	220	0.71s
4	SGD	0.01	64	(16,)	0.307	0.880	0.845	220	0.21s
5	SGD+momentum	0.001	16	(16,)	0.296	0.880	0.845	220	0.73s
6	SGD+momentum	0.01	16	(16,)	0.233	0.910	0.935	201	0.64s
7	SGD+momentum	0.05	16	(16,)	0.230	0.885	0.935	64	0.21s
8	SGD+momentum	0.01	64	(16,)	0.240	0.900	0.890	220	0.22s
9	Adam	0.001	16	(16,)	0.210	0.905	0.920	220	0.87s
10	Adam	0.01	16	(16,)	0.236	0.915	0.920	86	0.39s
11	Adam	0.05	16	(16,)	0.235	0.900	0.935	97	0.37s
12	Adam	0.01	64	(32,)	0.225	0.900	0.930	128	0.15s

Best validation configuration in this run set: Run 9, Adam, learning rate 0.001, batch size 16, hidden width (16,), validation loss 0.2102 and validation accuracy 0.905. Its final test accuracy is 0.920.

6.3 Optimization analysis

- A learning rate that is too small makes updates tiny and convergence slow within the epoch budget.
- A learning rate that is too large can overshoot minima, oscillate or produce unstable loss.
- SGD is simple but may converge slowly; momentum accumulates a smoothed direction and can accelerate progress; Adam adapts parameter-wise step sizes and often converges quickly.
- Smaller batches provide noisier but more frequent updates; larger batches give smoother gradients but fewer updates per epoch and may require learning-rate adjustment.

- A wider network increases capacity and parameter count, which can improve fit but increase training cost and overfitting risk.

The 12-run budget cannot exhaustively test all combinations or interactions. Conclusions are limited to the tested candidates and seeds. The test set must not guide tuning because repeated decisions based on test performance leak information and make the reported test score optimistically biased.

Q7 – Diagnose, Redesign and Validate a Neural Network Solution (13 Marks)

A. Diagnose the bottleneck (2 marks)

Selected limitation: a large 64–64 MLP trained on only 100 labelled examples has a high parameter-to-sample ratio (4,417 parameters for 100 samples), making it vulnerable to overfitting and unnecessary computational cost. The primary cause is limited labelled data relative to model capacity, rather than an inability to represent the nonlinear boundary.

B. Measurable design objective (2 marks)

Baseline: 64–64 hidden units, unregularized, 100 training examples. Redesign target: reduce parameter count by at least 70%, retain validation accuracy within 3 percentage points of the baseline (or improve it), and avoid a higher validation loss. Use the same train/validation/test partitions.

C. Revised learning strategy (4 marks)

Redesign: use a compact one-hidden-layer MLP with 16 ReLU units and L2 regularization $\alpha=0.02$. The model has $(2\times16+16)+(16\times1+1)=65$ parameters, compared with 4,417 for the 64–64 baseline. L2 reduces reliance on extreme weights, while the smaller model reduces variance and computational cost.

1. Split data into train / validation / test once.
2. Fit standardization on training data only.
3. Initialize compact MLP: 2 inputs -> 16 ReLU -> 1 sigmoid.
4. Train with Adam and L2 penalty (alpha = 0.02).
5. Monitor validation loss; retain best validation checkpoint.
6. Select using validation metrics; evaluate test once.

Trade-offs: the compact model may underfit if the true boundary is highly complex. L2 can also introduce bias. The redesign uses fewer parameters and less memory but does not guarantee better accuracy for every dataset.

D. Prediction before implementation (1 mark)

Prediction: parameter count will fall by more than 90%, training time will decrease, and the generalization gap should narrow. Validation accuracy should remain close to or exceed the baseline.

E. Implement and validate (2 marks)

Measure	**64–64 baseline**	**16-unit + L2 redesign**
Parameters	4417	65
Training accuracy	0.990	0.980
Validation accuracy	0.900	0.905
Validation loss	0.945	0.292
Test accuracy*	0.915	0.895
Epochs	187	321
Training time	0.243s	0.233s

*The selected redesign should be chosen from validation evidence; the test value is the final one-time estimate.

F. Final design justification (2 marks)

The redesign achieves the parameter-reduction objective by replacing 4,417 parameters with 65 (a reduction of approximately 98.5%). Whether it also meets the validation-accuracy target is determined by the table above. If validation loss or accuracy worsens beyond the stated tolerance, the hypothesis that a smaller regularized network generalizes better is not supported for this run; the compact model may underfit, and the original model may remain preferable when more labelled data are available. The final choice must be based on validation evidence, not the test score.

Overall Conclusion

The assignment demonstrates that model representation and training configuration are distinct. A single perceptron learns linearly separable AND but cannot learn XOR, regardless of learning rate. A nonlinear hidden layer enables an MLP to represent XOR. On a nonlinear two-feature dataset, shallow and deep networks should be compared under common data partitions and repeated seeds; greater depth is not automatically better. Limited labelled data increases overfitting risk, motivating L2, dropout, early stopping and capacity control. Hyperparameter tuning should use validation evidence and keep the test set untouched until final evaluation. A justified redesign links a measured limitation to a measurable objective and verifies the result under comparable conditions.

Appendix – Reproducible Python Implementation (Core snippets)

The experiments were produced with Python, NumPy, scikit-learn, Matplotlib and python-docx. The following snippets show the essential implementation patterns; the report tables and plots are generated from the same workflow.

A. Perceptron update

```
for x, target in zip(X, y):
    pred = int(np.dot(w, x) + b >= 0.5)
    error = target - pred
    if error:
        w += learning_rate * error * x
        b += learning_rate * error
```

B. NumPy backpropagation

```
h = sigmoid(X @ W1 + b1)
p = sigmoid(h @ W2 + b2)
d2 = (p - y) / len(y)
gW2 = h.T @ d2; gb2 = d2.sum(axis=0)
d1 = (d2 @ W2.T) * h * (1-h)
gW1 = X.T @ d1; gb1 = d1.sum(axis=0)
W1 -= lr*gW1; b1 -= lr*gb1
W2 -= lr*gW2; b2 -= lr*gb2
```

C. Dataset generation and leakage-safe scaling

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X, y = make_moons(n_samples=1000, noise=0.25, random_state=17)
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, test_size=0.40, random_state=17, stratify=y)
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

D. MLP training example

```
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(16,), activation='relu',
    solver='adam', alpha=0.02, learning_rate_init=0.01,
    batch_size=16, max_iter=500, random_state=21)
model.fit(X_train, y_train)
val_prob = model.predict_proba(X_val)[:, 1]
test_prob = model.predict_proba(X_test)[:, 1]  # only after selection
```

Note: scikit-learn's MLPClassifier does not expose a dropout option. The dropout comparison in Q5 was therefore implemented separately using NumPy with explicit inverted-dropout masks. The manual backpropagation question is also implemented directly in NumPy, as required by the assignment.